
Task W07: Sorting Algorithms

The Sorting Hat

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

1 Overview

This task has two required problems and two optional challenges. Problem 1 asks you to count the execution steps of a short method, the same way we did in lecture. Problem 2 asks you to perform four sorting algorithms, by hand, on a number set assigned to you. The two challenges at the end are not required and do not affect your grade in any way; they are there for anyone who wants to go further.

2 Diagram Submission Instructions

Some parts of this task ask for a diagram rather than typed text. You may choose one of the following options.

- **Hand-drawn on paper or a whiteboard.** Scan or take clear, well-lit photos. Images must be easy to read and clearly labeled with problem numbers. For this task in particular, this is the recommended option: several of these diagrams are easier to sketch by hand than to build in software.
- **Hand-drawn using an app** (for example Vittle Pro or Adobe Fresco). Save as an image file (PNG, JPG) or PDF. This is a convenient option but may be harder without a stylus.
- **Created with diagramming software** (for example Draw.io, LibreOffice Draw, or Excalidraw). Save in the tool's native file format; do not export as an image or PDF.
- **Other tools.** If you would like to use a different diagramming application, ask for approval first. Note that I have limited installation access on my office computer.

Include your name and the problem number on every diagram.

3 Problem 1: Counting Steps (5 points)

The method below reverses an array in place, swapping the first item with the last, the second item with the second to last, and so on, until it reaches the middle.

```

public static void reverse(int[] theArray, int n) {
01  for (int i = 0; i < n / 2; i++) {
02      int temp = theArray[i];
03      theArray[i] = theArray[n - 1 - i];
04      theArray[n - 1 - i] = temp;
05  }
06}

```

(a) Perform an algorithm analysis by counting the execution steps, as we did in the lectures. Show your work by listing the counts for each line and briefly explaining how you derived that count. Count as finely as possible. For each line, think through every individual operation the computer actually performs, every arithmetic operation, comparison, array access, and assignment, rather than treating the whole line as a single step. *Note: n is the length of the array.*

Hint: Think carefully about that loop.

Note: You can write the table out using the diagramming options above, or in the answer template provided with this task, instead of typing it in the space provided here.

For example (this is not from the code above, it is only to show the format):

Table 1: Example step-count table (not for the code above)

Line	Count	Explanation
01	n	initialized variable for each pass
02	$2n$	multiplied and added for each pass

Answer:

Table 2: Your step-count table for **reverse**

Line	Count	Explanation
01		
02		
03		
04		
05		
06		

(b) Write the summary expression for the step count. *Example: $4n^2 + 6n + 3$*

Answer:

(c) What is the Big-O notation for the algorithm? Briefly explain why, given your results from (a) and (b).

Answer:

4 Problem 2: Sorting by Hand (20 points)

Find your name in the table below and use that number set. Perform bubble sort, selection sort, mergesort, and quicksort on your assigned set, and show your work in the same style used in the example accompanying this document.

Note: Use the algorithm as it is presented in the textbook and in this course, not a variation you find elsewhere.

- **Bubble sort.** It can be done by diagram or typed out. You do not need to show the little arrows like in the lecture.
- **Selection sort.** It can be done by diagram or typed out. You do not need to show the little arrows like in the lecture.
- **Mergesort.** This should be a diagram showing arrows for splitting and merging.
- **Quicksort.** This should be a diagram showing arrows for splitting and a clear indicator of the pivot.

Note: If you type a sort out instead of diagramming it, submit it in a separate document with each problem clearly labeled, and use a monospace font, such as Courier, so the values stay aligned the same way they are in the example.

Table 3: Number sets for Problem 2

Name	Values
Seth	-10, 18, 46, 30, -6, 24, 42, 6, 28
Drake	-13, 34, -11, 23, -35, -7, 8, 33, -15
Korbin	-50, 13, -36, 1, -47, 25, 29, -39, -15
Davyn	-1, -38, -18, -48, 50, 25, 17, 36, 47
Zee	-44, 0, 16, -50, 8, 35, 13, 47, 7

5 What to Submit

- Problem 1, parts (a), (b), and (c).
- Problem 2, all four sorts, performed on your assigned number set.
- If you attempt either challenge below, include it as well, clearly labeled as a challenge.

6 Challenges (Not Graded)

Everything in this section is optional. Attempting it, or not attempting it, has no effect on your grade in this course. These are here for anyone who wants to go further with this material. If you attempt one, please consider submitting it; I would like to see what you tried, even if it is incomplete.

Challenge A: Radix Sort

Select one of the number sets below and perform the radix sort algorithm from our book, using the style shown in Figure 10-21 on page 546. You do not need to put the digits in bold, but you can if

it helps you keep track.

Table 4: Number sets for the radix sort challenge

Set	Values
1	5495, 2422, 2220, 6071, 2477, 8223, 7448, 8312, 4790
2	1606, 9421, 3204, 8127, 9138, 8677, 6007, 7231, 2625
3	6226, 5782, 9302, 6740, 1765, 7762, 8551, 7175, 5433
4	9504, 2767, 8508, 6991, 3460, 8063, 4223, 7546, 6112

Challenge B: Shell Sort and Heap Sort

Everything covered in this unit so far has been given to you as working code, or as an algorithm you traced first and coded second. This challenge works the other way. You are given two algorithms you have not seen in lecture, described below, and a single set of values. Trace each one by hand and produce a diagram, the same way you did for the sorts in Problem 2.

Values for this challenge: 62, -17, 34, 9, -45, 21, -3, 50, 15, -29, 6, 40

Shell sort

Shell sort is a variation on insertion sort. Ordinary insertion sort only ever compares an item to its immediate neighbor. Shell sort first compares items that are far apart, using a gap, then repeats the process with smaller and smaller gaps, until the gap is 1 and the final pass is an ordinary insertion sort. Sorting with a large gap first moves items that are badly out of place a long distance in a single pass, which ordinary insertion sort cannot do.

The gap sequence used here starts at $n/2$ (integer division) and is cut in half before each pass, stopping after the pass where the gap is 1.

```
public static void shellSort(int[] theArray, int n) {
01  for (int gap = n / 2; gap > 0; gap = gap / 2) {
02      for (int start = gap; start < n; start++) {
03          int nextItem = theArray[start];
04          int loc = start;
05          while (loc >= gap && theArray[loc - gap] > nextItem) {
06              theArray[loc] = theArray[loc - gap];
07              loc = loc - gap;
08          }
09          theArray[loc] = nextItem;
10      }
11  }
12}
```

For your diagram: show the array after each complete gap pass (that is, after the inner loop finishes for a given gap value), and label each row with the gap size used to produce it. You do not need to show every intermediate shift within a pass, the same way you were not required to show every comparison arrow for bubble and selection sort.

Heap sort

Heap sort treats the array as a complete binary tree, stored without any pointers: the item at index i has children at indexes $2i + 1$ and $2i + 2$, and, going the other direction, the item at index i

has a parent at index $(i - 1)/2$ (integer division). A *max heap* is an arrangement of the tree where every parent is greater than or equal to both of its children; note that this says nothing about how a node's two children compare to each other.

Heap sort has two phases. The first, **buildHeap**, rearranges the whole array into a max heap. The second phase repeatedly takes advantage of the fact that the largest item in a max heap is always at the root, index 0: it swaps the root with the last item in the unsorted part of the array, shrinks the unsorted part by one, and then repairs the heap property at the root with **siftDown**, since only the root is potentially out of place after that swap.

```
private static void siftDown(int[] theArray, int root, int heapSize) {
01  int largest = root;
02  int left = 2 * root + 1;
03  int right = 2 * root + 2;
04  if (left < heapSize && theArray[left] > theArray[largest]) {
05      largest = left;
06  }
07  if (right < heapSize && theArray[right] > theArray[largest]) {
08      largest = right;
09  }
10  if (largest != root) {
11      int temp = theArray[root];
12      theArray[root] = theArray[largest];
13      theArray[largest] = temp;
14      siftDown(theArray, largest, heapSize);
15  }
16}

public static void heapSort(int[] theArray, int n) {
17  for (int root = n / 2 - 1; root >= 0; root--) {
18      siftDown(theArray, root, n);
19  }
20  for (int last = n - 1; last >= 1; last--) {
21      int temp = theArray[0];
22      theArray[0] = theArray[last];
23      theArray[last] = temp;
24      siftDown(theArray, 0, last);
25  }
26}
```

For your diagram: draw the array as a binary tree, the same way it is described above, at each of these points: the starting arrangement, after **buildHeap** completes (lines 17 through 19), and after each root-and-last swap in the second phase (lines 20 through 25), before that item's **siftDown** repairs the tree. Use arrows to show each swap. You do not need to diagram every internal step of a single **siftDown** call, only the state of the tree right before and right after it runs.